

HW 3

Sam Ly

February 14, 2026

Chapter 6

The code for the challenge questions can be found here:

<https://github.com/samlyme/Assignments/tree/cs4080/ch6-challenge>

1. In C, a block is a statement form that allows you to pack a series of statements where a single one is expected. The comma operator is an analogous syntax for expressions. A comma-separated series of expressions can be given where a single expression is expected (except inside a function call's argument list). At runtime, the comma operator evaluates the left operand and discards the result. Then it evaluates and returns the right operand.

Add support for comma expressions. Give them the same precedence and associativity as in C. Write the grammar, and then implement the necessary parsing code.

First, to create the grammar, we define a new lowest precedence below `equality`. So, the new grammar becomes:

```
expression    → comma;      // CHALLENGE!!
comma         →> equality ("," equality)*;
equality      → comparison ( ( "!=" | "==" ) comparison )* ;
comparison    → term ( ( ">" | ">=" | "<" | "<=" ) term )* ;
term          → factor ( ( "-" | "+" ) factor )* ;
factor        → unary ( ( "/" | "*" ) unary )* ;
unary         → ( "!" | "-" ) unary
              | primary ;
primary       → NUMBER | STRING | "true" | "false" | "nil"
              | "(" expression ")" ;
```

To implement this, we override the current `Expr` method to match for the `Comma` token first.

```
private Expr expression() {
    return comma();
}

private Expr comma() {
    Expr expr = equality();

    while (match(COMMA)) {
        Token op = previous();
        Expr right = equality();
        expr = new Expr.Binary(expr, op, right);
    }
    return expr;
}
```

2. Likewise, add support for the C-style conditional or “ternary” operator `?:`. What precedence level is allowed between the `?` and `:`? Is the whole operator left-associative or right-associative?

The ternary operator has slightly higher precedence than the comma. This is because `(a ? b : c, d, e)` should evaluate to `(b, d, e)` or `(c, d, e)`.

Although the C-style ternary can be nested and is right-associative, in my implementation I chose to explicitly enforce the use of parens when nesting ternaries. The implementation also required creating a new Expression type since the ternary expression is actually a trinary expression.

```
private Expr ternary() {
    Expr expr = equality();

    if (match(QUESTION)) {
        Token leftOp = previous();
        Expr center = equality();

        consume(COLON, "Expect ':' in ternary.");
        Token rightOp = previous();
        Expr right = equality();

        expr = new Expr.Trinary(expr, leftOp, center, rightOp, right);
    }

    return expr;
}
```

3. Add error productions to handle each binary operator appearing without a left-hand operand. In other words, detect a binary operator appearing at the beginning of an expression. Report that as an error, but also parse and discard a right-hand operand with the appropriate precedence.

The grammar for error productions now looks like this:

```
expression    → comma;          // CHALLENGE!!
comma         → ternary ("," ternary)*;
ternary       → ternary "?" ternary ":" ternary;
               | equality;
equality      → comparison? ( ( "!=" | "==" ) comparison )* ;
comparison    → term? ( ( ">" | ">=" | "<" | "<=" ) term )* ;
term          → factor? ( ( "-" | "+" ) factor )* ;
factor        → unary? ( ( "/" | "*" ) unary )* ;
unary         → ( "!" | "-" ) unary
               | primary ;
primary       → NUMBER | STRING | "true" | "false" | "nil"
               | "(" expression ")" ;
```

However, when implementing, we defer the check all the down to the `primary` method because it allows us to handle it as an error.

```
private Expr primary() {
    if (match(FALSE)) return new Expr.Literal(false);
    if (match(TRUE)) return new Expr.Literal(true);
    if (match(NIL)) return new Expr.Literal(null);

    if (match(NUMBER, STRING)) {
        return new Expr.Literal(previous().literal);
    }
}
```

```

    }

    if (match(LEFT_PAREN)) {
        Expr expr = expression();
        consume(RIGHT_PAREN, "Expect ')' after expression.");
        return new Expr.Grouping(expr);
    }

    if (match(PLUS, MINUS, STAR, SLASH,
              BANG_EQUAL, EQUAL_EQUAL, GREATER,
              GREATER_EQUAL, LESS, LESS_EQUAL)) {
        error(previous(), "Missing left operand");
        term();
        return null;
    }

    throw error(peek(), "Expect expression.");
}

```

Chapter 7

1. Allowing comparisons on types other than numbers could be useful. The operators might have a reasonable interpretation for strings. Even comparisons among mixed types, like `3 < "pancake"` could be handy to enable things like ordered collections of heterogeneous types. Or it could simply lead to bugs and confusion.

Would you extend Lox to support comparing other types? If so, which pairs of types do you allow and how do you define their ordering? Justify your choices and compare them to other languages.

I personally would not extend Lox to support comparing other types for two reasons: 1) ambiguity and unexpected behavior. Lox is already dynamically typed, so incorporating this could lead to cursed situations where you need to reason about the hypothetical behavior of an operator based on the hypothetical type of its operands. 2) implementation difficulty. There is no good way to perform dynamic dispatch at this level. Sure, we can override each ‘internal’ type to have a custom comparing method, but that method would then need to somehow infer the ‘absolute value’ of arbitrary objects. A good example of this is Java itself! Java explicitly does not allow for comparing different types.

2. Many languages define `+` such that if either operand is a string, the other is converted to a string and the results are then concatenated. For example, `"scone" + 4` would yield `scone4`. Extend the code in `visitBinaryExpr()` to support that.

```

case PLUS:
    if (left instanceof Double && right instanceof Double) {
        return (double)left + (double)right;
    }
    if (left instanceof String || right instanceof String) {
        return stringify(left) + stringify(right);
    }
    throw new RuntimeError(expr.operator,
        "Operands must be two numbers or two strings.");

```

3. What happens right now if you divide a number by zero? What do you think should happen? Justify your choice. How do other languages you know handle division by zero, and why do they make the choices they do?

Change the implementation in `visitBinaryExpr()` to detect and report a runtime error for this case.

Now, if we divide by zero, we get ‘Infinity’. This is a natural consequence of Java’s implementation of the `Double` class. I think this actually makes sense! It gives a good way for the program to gracefully handle edge cases that is also obvious for users to debug. Some other languages either return a special ‘NaN’ value or throw an exception. These choices also make sense. By returning ‘NaN’, the downstream mathematical operations can be halted, and also gives the user a good idea of the source of the bug. By throwing an exception, the user is forced to handle the edge case.

```
case SLASH:
    checkNumberOperands(expr.operator, left, right);
    if (right.equals(0.0))
        throw new RuntimeException(
            expr.operator, "Can't divide by zero.");
    return (double)left / (double)right;
```